UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 11

Funciones Recursivas

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 La recursión antes de las computadoras **2**

2 ¿Qué es una función recursiva? **3**

3 El primer ejemplo: suma acumulada **3**
3.1 Traza de `suma(4)` 3

4 El call stack: la memoria detrás de la recursión **4**
4.1 La pila durante `suma(3)` 4

5 Sin caso base: stack overflow **5**

6 Ejemplos adicionales **5**
6.1 Factorial 5
6.2 Potencias 6
6.3 Sucesión de Fibonacci 6

7 Recursivo vs. iterativo **6**

8 Ejercicios **7**

9 Resumen **7**

La recursión antes de las computadoras

Mucho antes de que existiera C++, o cualquier lenguaje de programación, los matemáticos ya llevaban siglos usando la recursión sin llamarla así.

En 1202, el matemático italiano **Leonardo de Pisa** —conocido como Fibonacci— publicó su *Liber Abaci* con el siguiente problema:

> *"Un par de conejos produce otro par cada mes. Los conejos tardan un mes en madurar y, a partir del segundo mes, producen un par nuevo cada mes. ¿Cuántos pares habrá después de un año?"*

La solución es la célebre **sucesión de Fibonacci**: $1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$ donde cada número es la suma de los dos anteriores.

Escrito formalmente:

$$F(1) = 1 \qquad F(2) = 1 \qquad F(n) = F(n-1) + F(n-2)$$

Eso *es* una definición recursiva: un valor definido en términos de versiones anteriores de sí mismo, con casos base que detienen la cadena. Los matemáticos usan este estilo desde hace siglos; nosotros simplemente lo vamos a traducir a código.

En 1931 el matemático **Kurt Gödel** usó lo que hoy llamamos *funciones recursivas primitivas* en su célebre teorema de incompletitud para codificar demostraciones matemáticas como números. La idea de que una función puede definirse en términos de sí misma con un rango más pequeño fue formalizada por Gödel mucho antes de que hubiera procesadores capaces de ejecutarla.
Referencia: Gödel, K. *Über formal unentscheidbare Sätze der Principia Mathematica*. Monatshefte für Mathematik, 1931.

El primer lenguaje de programación que soportó funciones recursivas de forma nativa fue **LISP**, diseñado por **John McCarthy** en el MIT en 1958. McCarthy quería un lenguaje capaz de procesar listas de manera similar a como los matemáticos definen funciones: en términos de sí mismas. Antes de LISP, los lenguajes como FORTRAN no tenían una pila de llamadas real y la recursión *simplemente no era posible*.
Referencia: McCarthy, J. *Recursive Functions of Symbolic Expressions and Their Computation by Machine*. CACM, 1960.

¿Qué es una función recursiva?

Una **función recursiva** es una función que, para resolver un problema, se llama a sí misma con una versión *más pequeña* del mismo problema.

La clave es esa palabra: **más pequeña**. No puede llamarse con el mismo problema (recursión infinita) ni con uno más grande (divergencia). Cada llamada debe acercar el problema a un punto donde la respuesta sea inmediata y no requiera más llamadas.

Parte	Descripción
Caso base	La versión del problema tan pequeña que se resuelve directamente, sin más
Caso recursivo	Reduce el problema un paso y delega el resto a una llamada con un argume

Si falta el **caso base**, la función se llama infinitamente y el programa termina con un **stack overflow** (desbordamiento de pila). Si el caso recursivo **no reduce** el problema, pasa lo mismo. Ambos errores son los más comunes al aprender recursión.

El primer ejemplo: suma acumulada

Queremos calcular $1 + 2 + 3 + \cdots + n$ de forma recursiva. La observación clave es:

$$\text{suma}(n) = n + \underbrace{(1 + 2 + \cdots + (n-1))}_{\text{suma}(n-1)}$$

Con eso, la definición completa es:

Caso	Definición
$n = 0$ (caso base)	$\text{suma}(0) = 0$
$n > 0$ (caso recursivo)	$\text{suma}(n) = n + \text{suma}(n-1)$

- **Traza de `suma(4)`**

Llamada	Expansión
`suma(4)`	$= 4 +$ `suma(3)`
`suma(3)`	$= 3 +$ `suma(2)`
`suma(2)`	$= 2 +$ `suma(1)`
`suma(1)`	$= 1 +$ `suma(0)`
`suma(0)`	$= 0$ *(caso base)*
Regresando:	
`suma(1)`	$= 1 + 0 = 1$
`suma(2)`	$= 2 + 1 = 3$
`suma(3)`	$= 3 + 3 = 6$
`suma(4)`	$= 4 + 6 = 10$

Para trazar una función recursiva a mano, dibuja primero todas las llamadas "hacia abajo" hasta el caso base, y luego sustituye los valores "hacia arriba". El árbol tiene forma de V: baja hasta el caso base y sube resolviendo.

El call stack: la memoria detrás de la recursión

Cada vez que una función se llama, el sistema reserva un bloque de memoria llamado **stack frame** (marco de pila). Ese bloque guarda:

- Los parámetros recibidos (`x = 4`, `x = 3`, ...)
- Las variables locales
- La **dirección de retorno**: a qué línea regresar cuando la función termine

Los frames se **apilan** conforme se hacen llamadas y se **desapilan** en orden inverso al regresar. Este mecanismo es el mismo que viste en la sesión 01 con la analogía del escritorio: la *RAM* acumula los frames como platos en una pila.

- **La pila durante `suma(3)`**

```
Estado JUSTO ANTES de que suma(0) retorne:

+-------------------------+  <- tope (llamada más reciente)
|  suma(0)   x = 0        |  retorna 0 de inmediato
+-------------------------+
|  suma(1)   x = 1        |  espera el resultado de suma(0)
+-------------------------+
|  suma(2)   x = 2        |  espera el resultado de suma(1)
+-------------------------+
|  suma(3)   x = 3        |  espera el resultado de suma(2)
+-------------------------+
|  main()                 |  espera el resultado de suma(3)
```

```
+-------------------------+  <- fondo

Despues, los frames se desapilan:
suma(0)->0   suma(1)->1   suma(2)->3   suma(3)->6   main imprime 6
```

✔ Recuerda

La profundidad de la recursión (número de frames apilados al mismo tiempo) está limitada por el tamaño de la pila del proceso, que en la mayoría de sistemas es **1–8 MB**. Para `suma(n)`, la profundidad máxima es $n + 1$ frames. Con $n = 100{,}000$ es posible llegar al límite.

Sin caso base: stack overflow

Si se omite el caso base, la pila crece sin detenerse:

```
int sumaSinBase(int x) {
    return x + sumaSinBase(x - 1);   // <- nunca para
}

  sumaSinBase(10)
  sumaSinBase(9)
  sumaSinBase(8)
  ...
  sumaSinBase(-50000)
  ...
  CRASH!  Segmentation fault / Stack overflow
```

El nombre del sitio de preguntas técnicas más famoso del mundo, **Stack Overflow** (`https://stackoverflow.com`), es exactamente esta situación: la broma interna del fundador Jeff Atwood sobre el error más icónico al aprender a programar.

Ejemplos adicionales

- **Factorial**

$$0! = 1 \qquad n! = n \times (n-1)!$$

$$5! = 5\times4! = 5\times4\times3! = 5\times4\times3\times2! = 5\times4\times3\times2\times1! = 5\times4\times3\times2\times1\times0! = 120$$

El factorial crece extremadamente rápido: $13! > 6 \times 10^9$, que ya supera el rango de un `int` de 32 bits. Usa `long long` para factoriales mayores que 12.

- **Potencias**

$$b^0 = 1 \qquad b^n = b \times b^{n-1}$$

$$2^5 = 2 \times 2^4 = 2 \times 2 \times 2^3 = \cdots = 2^5 = 32$$

Cada llamada reduce el exponente en 1 hasta llegar a $b^0 = 1$.

- **Sucesión de Fibonacci**

$$F(1) = 1, \quad F(2) = 1, \quad F(n) = F(n-1) + F(n-2)$$

Fibonacci es uno de los ejemplos más naturales de recursión **doble**: el caso recursivo hace *dos* llamadas en lugar de una.

La implementación recursiva directa de Fibonacci es **ineficiente**: $F(40)$ requiere más de 10^8 llamadas porque calcula los mismos valores una y otra vez. Este problema se resuelve con *memoización* o programación dinámica — temas de cursos posteriores.

La sucesión de Fibonacci aparece en la disposición de las semillas de un girasol, las espirales de las piñas de pino y el número de pétalos de muchas flores. La razón $F(n+1)/F(n)$ converge a la **proporción áurea** $\varphi \approx 1.618\ldots$, que también aparece en la arquitectura griega y en el arte del Renacimiento. Lo que comenzó como un problema de conejos en 1202 terminó siendo una de las constantes matemáticas más observadas en la naturaleza.
Referencia: Livio, M. *The Golden Ratio*. Broadway Books, 2002.

Recursivo vs. iterativo

Cualquier función recursiva puede reescribirse con un ciclo, y viceversa. La elección depende de la claridad y el contexto.

Criterio	Recursivo	Iterativo
Legibilidad	Alta cuando el problema es naturalmente recursivo	Alta cuando hay un ciclo
Memoria	Un frame por llamada activa (pila)	Solo las variables del cicl
Riesgo de fallo	Stack overflow si la profundidad es grande	No tiene este riesgo
Casos ideales	Árboles, backtracking, divide y vencerás	Sumas, búsquedas simpl

✔ Recuerda

En este curso veremos que los algoritmos sobre **árboles** y **grafos** son casi siempre más elegantes de forma recursiva. Las **Torres de Hanoi** de la sesión 12 son el ejemplo clásico donde la solución iterativa equivalente es considerablemente más difícil de entender.

Ejercicios

Resuelve los siguientes ejercicios **a mano, en papel**. Para cada función recursiva, muestra la traza completa de llamadas (caso base y regreso de valores).

1. Calcula `suma(6)` trazando todas las llamadas. ¿Cuántos frames se apilan simultáneamente como máximo?
2. Calcula `factorial(6)` trazando la expansión y la resolución inversa. ¿Cuántas multiplicaciones se realizan?
3. Calcula `pot(3, 4)` paso a paso. Verifica que el resultado sea 81.
4. La siguiente función tiene un error. Identifica el problema, explica qué pasaría en ejecución y escribe la versión corregida.

```
int malo(int n) { return n + malo(n + 1); }
```

5. Escribe la **definición recursiva** (en notación matemática, no en código) de las siguientes funciones:
   a) La suma de los primeros n números impares: $1 + 3 + 5 + \cdots + (2n - 1)$.
   b) El producto $n \times (n + 1) \times (n + 2) \times \cdots \times m$ (con $n \leq m$).
6. Fibonacci. Calcula $F(7)$ trazando el árbol de llamadas. Cuenta cuántas veces se calcula $F(3)$ en el proceso. ¿Ves el problema de eficiencia?
7. **Problema de reflexión.** La función `suma(n)` tiene profundidad de recursión $n + 1$. Si el sistema admite una pila de 8 MB y cada frame ocupa 32 bytes, ¿cuál es la profundidad máxima teórica? ¿Qué tan grande puede ser n antes de provocar stack overflow?

Resumen

✔ Recuerda

- Una función recursiva se llama a sí misma con un argumento **más pequeño**; cada llamada se acerca al caso base.
- Toda función recursiva correcta tiene: (1) un **caso base** que retorna sin llamar, y (2) un **caso recursivo** que reduce el problema.
- Cada llamada crea un **stack frame** en la pila; los frames se desapilan al retornar, en orden inverso.

- Sin caso base → **stack overflow**. Si el argumento no decrece → también.
- La recursión es más elegante que los ciclos en problemas con estructura naturalmente recursiva (árboles, particiones, etc.).
- Fibonacci recursivo directo es **exponencialmente ineficiente**; hay soluciones mejores que veremos más adelante.